A Formal Description of Algorithms

In *Sec.* 5.2, we propose the Selection Cache, which shares selection results among similar Queries to reduce selection frequency without sacrificing performance. Formally, it is defined as follows:

Algorithm 1 Selection Cache Algorithm

Require: $\mathbf{Q} \in \mathbb{R}^{H \times D}$: current query vectors
$k \in \mathbb{N}$: number of tokens to select
$\mathbf{C}_Q \in \mathbb{R}^{H \times D}$: cached query vector
$\mathbf{C}_\mathcal{I} \in \{0, \ldots, N-1\}^k$: cached indices
$\theta \in [0, 1]$: cosine-similarity threshold
$\mathcal{S}$: selection function (Eq. 7)
$f \in \{\text{True}, \text{False}\}$: first-query flag (default True)
Ensure: $\mathcal{I} \in \{0, \ldots, N-1\}^k$: indices of k selected tokens

```
if f or cos(Q, C_Q) < θ then
    I ← S(Q, k)
    C_I ← I
    C_Q ← Q
    f ← False
else
    I ← C_I
end if
return I
```

In *Sec.* 5.3, we propose the Paged Dot Product Kernel to efficiently perform token-level per-head criticality estimation under the paged KV-cache management by significantly reducing I/O between HBM and SRAM. Formally, it is defined as follows:

Algorithm 2 Paged Dot Product Kernel

Require: $\mathbf{Q} \in \mathbb{R}^{H \times D}$: current query vectors
$\mathbf{K} \in \mathbb{R}^{N_{\text{kv}} \times H_{\text{kv}} \times D}$: key cache pool
$\mathbf{I} \in \{0, \ldots, N_{\text{kv}} - 1\}^T$: indices of relevant tokens
H: number of attention heads
H_{kv}: number of KV heads ($H \bmod H_{\text{kv}} = 0$)
D: head dimension
T: number of relevant tokens ($|\mathbf{I}| = T$)
B: CUDA block size
Ensure: $\mathbf{S} \in \mathbb{R}^{H \times T}$: dot product scores

```
N ← ⌈T/B⌉
for all h = 0, ..., H − 1 in parallel do
    q ← Q[h, :] {to SRAM}
    h_kv ← h mod H_kv
    for all b = 0, ..., N − 1 in parallel do
        t_0 ← b × B
        L ← min(B, T − t_0)
        for j = 0, ..., L − 1 do
            idx ← I[t_0 + j] {to SRAM}
            k ← K[idx, h_kv, :] {to SRAM}
            s ← ⟨q, k⟩ {in SRAM}
            S[h, t_0 + j] ← s {to HBM}
        end for
    end for
end for
return S
```

B Scalability of *TokenSelect*

B.1 Scaling Beyond 1 Million Context Length

To further explore *TokenSelect*'s performance in extreme long-context scenarios, we design an extended benchmark with different text lengths following InfiniteBench. As illustrated in the *Fig.* 9, our *TokenSelect* demonstrates the ability to accurately capture critical information with a small token budget in contexts up to 2M tokens, underscoring its potential in more application scenarios.

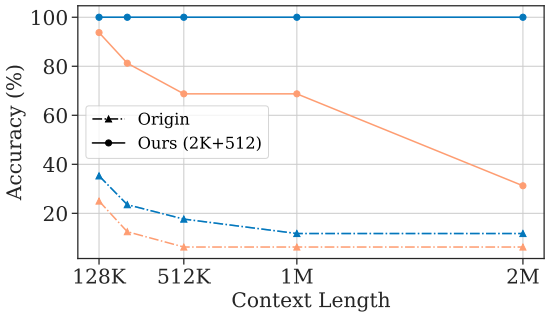

Figure 9: Performance comparison on extended R.PK and R.KV using `Qwen2-7B-Instruct`.

B.2 Scaling to 72 Billion Parameters

To demonstrate the scalability of our approach to larger models, we conducted additional experiments using `Qwen2-72B-Instruct`. The results, presented in Table 6, show that our method outperforms NTK-Aware Scaled RoPE in terms of accuracy and achieves lower latency, indicating the potential of our approach to scale effectively with larger models.

Method	En.Sum		En.QA		R.KV	
	Acc. (%)	Time (s)	Acc. (%)	Time (s)	Acc. (%)	Time (s)
NTK (SPDA)	23.49	199.52	28.77	145.69	50.00	111.98
TokenSelect	**25.07**	**114.24**	**29.91**	**71.98**	**88.12**	**63.27**

Table 6: Performance and latency comparison on `Qwen2-72B-Instruct` with tensor parallelism size: 4.

C Formal Statement and Proof of Lemma

Lemma 1 (Invariant Top-k Key Selection under Cosine Similarity Threshold, Formal).

Assumptions:

1. Let $\mathbf{q}_1, \mathbf{q}_2 \in \mathbb{R}^d$ be two query vectors.
2. Let $\{\mathbf{k}_i\}_{i=1}^N \subset \mathbb{R}^d$ be a finite set of key vectors.
3. Let k be a positive integer such that $1 \leq k \leq N$.
4. Define the cosine similarity between vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ as:
$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{ab}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2},$$
where $\|\cdot\|_2$ denotes the Euclidean norm.
5. Define the top-k selection function based on dot product similarity as: $\mathcal{I}(\mathbf{q}) = \arg\max_{S \subseteq \{1,2,\ldots,N\}, |S|=k} \sum_{i \in S} \mathbf{q} \cdot \mathbf{k}_i$. Assume that for any query vectors $\mathbf{q}$, the top-k set $\mathcal{I}(\mathbf{q})$ is uniquely determined.
6. Let $\epsilon \in (0, 1]$ be a predefined threshold.

Lemma Statement: *If the cosine similarity between the two query vectors $\mathbf{q}_1$ and $\mathbf{q}_2$ satisfies*

$$\cos(\mathbf{q}_1, \mathbf{q}_2) > \epsilon,$$

then the indices of the top-k keys selected by $\mathbf{q}_1$ and $\mathbf{q}_2$ are identical, i.e.,

$$\mathcal{I}(\mathbf{q}_1) = \mathcal{I}(\mathbf{q}_2).$$

Proof: We start with the given condition:

$$\min_{1 \leq i \leq k} \mathbf{q}_1 \mathbf{k}_i - \max_{j > k} \mathbf{q}_1 \mathbf{k}_j > \eta,$$

which we aim to use to demonstrate that:

$$\min_{1 \leq i \leq k} \mathbf{q}_2 \mathbf{k}_i - \max_{j > k} \mathbf{q}_2 \mathbf{k}_j > 0.$$

To facilitate our analysis, we introduce the following notations:

$$\hat{\eta} = \frac{\eta}{\|\mathbf{q}_1\|}, \quad \hat{\mathbf{q}}_1 = \frac{\mathbf{q}_1}{\|\mathbf{q}_1\|}, \quad \hat{\mathbf{q}}_2 = \frac{\mathbf{q}_2}{\|\mathbf{q}_2\|}.$$

With these definitions, the original condition becomes:

$$\min_{1 \leq i \leq k} \hat{\mathbf{q}}_1 \mathbf{k}_i - \max_{j > k} \hat{\mathbf{q}}_1 \mathbf{k}_j > \hat{\eta},$$

and our goal transforms to showing:

$$\min_{1 \leq i \leq k} \hat{\mathbf{q}}_2 \mathbf{k}_i - \max_{j > k} \hat{\mathbf{q}}_2 \mathbf{k}_j > 0.$$

Next, let θ denote the angle between $\mathbf{q}_1$ and $\mathbf{q}_2$, $\cos\theta = \hat{\mathbf{q}}_1 \cdot \hat{\mathbf{q}}_2$. We can further define:

$$\mathbf{p}_1 = \mathbf{q}_2 - \mathbf{q}_1 \cos\theta, \quad \hat{\mathbf{p}}_1 = \frac{\mathbf{p}_1}{\|\mathbf{p}_1\|},$$

then $\sin\theta = \hat{\mathbf{p}}_1 \cdot \hat{\mathbf{q}}_2$, and

$$\hat{\mathbf{q}}_2 = \hat{\mathbf{q}}_1 \cos\theta + \hat{\mathbf{p}}_1 \sin\theta.$$

Then we have:

$$\begin{aligned}
\min_{1 \leq i \leq k} \hat{\mathbf{q}}_2 \mathbf{k}_i &= \min_{1 \leq i \leq k} (\hat{\mathbf{q}}_1 \cos\theta + \hat{\mathbf{p}}_1 \sin\theta) \mathbf{k}_i, \\
&\geq \min_{1 \leq i \leq k} \hat{\mathbf{q}}_1 \mathbf{k}_i \cos\theta + \min_{1 \leq i \leq k} \hat{\mathbf{p}}_1 \mathbf{k}_i \sin\theta, \\
&\geq \hat{\mathbf{q}}_1 \mathbf{k}_k \cos\theta - \|\mathbf{k}\|_{\max} \sin\theta,
\end{aligned}$$

and

$$\begin{aligned}
\max_{j > k} \hat{\mathbf{q}}_2 \mathbf{k}_j &= \max_{j > k} (\hat{\mathbf{q}}_1 \cos\theta + \hat{\mathbf{p}}_1 \sin\theta) \mathbf{k}_j \\
&\leq \max_{j > k} \hat{\mathbf{q}}_1 \mathbf{k}_i \cos\theta + \max_{j > k} \hat{\mathbf{p}}_1 \mathbf{k}_i \sin\theta, \\
&\leq \hat{\mathbf{q}}_1 \mathbf{k}_{p+1} \cos\theta + \|\mathbf{k}\|_{\max} \sin\theta.
\end{aligned}$$

Therefore,

$$\begin{aligned}
&\min_{1 \leq i \leq k} \hat{\mathbf{q}}_2 \mathbf{k}_i - \max_{j > k} \hat{\mathbf{q}}_2 \mathbf{k}_j \\
&\qquad \geq \hat{\mathbf{q}}_1 \mathbf{k}_p \cos\theta - \|\mathbf{k}\|_{\max} \sin\theta \\
&\qquad\qquad - (\hat{\mathbf{q}}_1 \mathbf{k}_{p+1} \cos\theta \\
&\qquad\qquad + \|\mathbf{k}\|_{\max} \sin\theta) \\
&\qquad = (\hat{\mathbf{q}}_1 \mathbf{k}_p \cos\theta - \hat{\mathbf{q}}_1 \mathbf{k}_{p+1} \cos\theta) \\
&\qquad\qquad - 2\|\mathbf{k}\|_{\max} \sin\theta \\
&\qquad \geq \hat{\eta} \cos\theta - 2\|\mathbf{k}\|_{\max} \sin\theta. \qquad (8)
\end{aligned}$$

In order to have Eqn. (8) > 0, we require

$$\begin{aligned}
&\hat{\eta} \cos\theta > 2\|\mathbf{k}\|_{\max} \sin\theta, \\
\Rightarrow\ &\frac{\sin\theta}{\cos\theta} < \frac{\hat{\eta}}{2\|\mathbf{k}\|_{\max}}, \\
\Rightarrow\ &\frac{1 - \cos^2\theta}{\cos^2\theta} < \left(\frac{\hat{\eta}}{2\|\mathbf{k}\|_{\max}}\right)^2, \\
\Rightarrow\ &\cos\theta \geq \frac{1}{\sqrt{1 + \left(\frac{\hat{\eta}}{2\|\mathbf{k}\|_{\max}}\right)^2}}.
\end{aligned}$$

This final inequality establishes a sufficient condition for the original statement to hold, thereby completing the proof.

D Overview of LLMs Inference

Nowadays, mainstream LLMs are primarily based on the Decoder-only Transformer architecture. Each transformer layer includes a multi-head attention (MHA) and a feed-forward networks (FFN). The inference process of LLMs can be divided into two stages: the prefill stage and the decode stage.

The prefill stage is the preparatory phase of the inference process. In this stage, the user's input is processed layer by layer through a single forward pass of LLMs, generating KV Cache for each layer. The generation of KV Cache is completed by the MHA module. Assuming $\mathbf{X}_{\text{prefill}} \in \mathbb{R}^{n_{\text{in}} \times d}$ is the input of a transformer layer, where n_{in} is the number of tokens in user's input sequence and d is the hidden size. The MHA computation in the prefill stage is as follows (simplified to single head):

$$[\mathbf{Q}_{\text{prefill}}, \mathbf{K}_{\text{prefill}}, \mathbf{V}_{\text{prefill}}] = \mathbf{X}_{\text{prefill}} \cdot [\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v], \quad (9)$$

$$\mathbf{O}_{\text{prefill}} = \text{softmax}\left(\frac{\mathbf{Q}_{\text{prefill}} \cdot \mathbf{K}_{\text{prefill}}^{\top}}{\sqrt{d}}\right) \cdot \mathbf{V}_{\text{prefill}}, \quad (10)$$

where $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$ are linear projections, $[\cdot]$ represents tensor concatenation operation, and *Eq.*(10) is also known as Scaled Dot Product Attention (SDPA). After these computation, $\mathbf{K}_{\text{prefill}}$ and $\mathbf{V}_{\text{prefill}}$ are stored as the KV Cache for current layer $\mathbf{K}_{\text{cache}}$ and $\mathbf{V}_{\text{cache}}$, and $\mathbf{O}_{\text{prefill}}$ is used for subsequent calculations.

The decode stage is the phase where LLMs actually generate the response. In the decode stage, LLMs load the KV Cache and generate n_{out} output tokens autoregressively through n_{out} forward passes. Assuming $\mathbf{X}_{\text{decode}} \in \mathbb{R}^{1 \times d}$ is the input of a transformer layer in a forward pass, the computation of MHA in the decode stage is as follows (The calculation of $\mathbf{Q}_{\text{prefill}}$ and $\mathbf{O}_{\text{prefill}}$ is consistent with that in the prefill stage):

$$\begin{aligned} \mathbf{K}_{\text{decode}} &= [\mathbf{K}_{\text{cache}},\ \mathbf{X}_{\text{decode}} \cdot \mathbf{W}_k],\ \mathbf{K}_{\text{cache}} \leftarrow \mathbf{K}_{\text{decode}}, \\ \mathbf{V}_{\text{decode}} &= [\mathbf{V}_{\text{cache}},\ \mathbf{X}_{\text{decode}} \cdot \mathbf{W}_v],\ \mathbf{V}_{\text{cache}} \leftarrow \mathbf{V}_{\text{decode}}, \end{aligned} \quad (11)$$

where $\mathbf{K}_{\text{decode}}, \mathbf{V}_{\text{decode}}$ are composed of the KV Cache and the KV corresponding to the current input, which are then used to update the KV Cache of the current layer for use in the next forward pass.

LLMs inference, unlike training, is memory-bound, necessitating frequent GPU I/O operations between HBM and SRAM while underutilizing processing units. This bottleneck is particularly evident in SDPA computation. Optimizing for I/O is crucial for enhancing LLMs inference efficiency, especially in long-context scenarios.

E Comparison with Token Eviction-based Methods (*e.g.*, H_2O)

Token eviction–based methods (Zhang et al., 2024b; Oren et al., 2024), led by H_2O (Zhang et al., 2024b), have pioneered the field of long-context inference, achieving early state-of-the-art performance. Although both our method and H_2O employ token-level criticality estimation, they fall under two entirely different taxonomies. As discussed in *Sec.* 2 and *Sec.* 3, H_2O is a query-independent KV cache selection method, which suffers from three main drawbacks:

1. Lack of dynamism: Its importance scoring relies on attention scores from previous queries and keys. Consequently, KV pairs that are crucial for the current query may have been discarded earlier—a phenomenon also confirmed by QUEST (Tang et al., 2024). *Fig.* 1 and 2 of QUEST provide an intuitive illustration of the differences between query-based methods (e.g., our *TokenSelect*) and H_2O. Notably, *TokenSelect* leverages a dynamic selection strategy, enabling state-of-the-art performance with a minimal token budget.

2. Inability to extend sequence length: Since H_2O depends on the model's original attention mechanism, it cannot extend the effective context length. In contrast, our approach can easily extend a model with an original maximum length of 4K–32K tokens to an effective length exceeding 1M tokens.

3. Inefficient implementation: H_2O evaluates token importance based on attention scores, making it incompatible with efficient kernels such as FlashAttention (Dao, 2024). This limitation restricts its scalability. Our method, however, is designed for broad compatibility and is fully transparent to large-scale inference acceleration infrastructures, including paged attention, tensor parallelism, and prefix caching, making it ready for large-scale online serving.

To further demonstrate the superiority of *TokenSelect*, we present experimental results in Table 7. These results corroborate the findings of previous studies (Tang et al., 2024; Xiao et al., 2024a), showing that query-independent methods are inferior to query-based approaches.